TECHNICAL REPORT · MACHINE LEARNING · NLP

Fake News Detector

Bidirectional LSTM text classification: design, training, evaluation and a data-leakage audit

TensorFlow / Keras | BiLSTM | Streamlit | Python | MIT License

Author: Mahan Liaghatmand · Report date: October 2026 · Version 1.0

5,829	2.66 M	99.66%*	99.6%
labeled news articles	trainable parameters	test accuracy (see leakage audit)	of Real articles contain "Reuters"

About this report

This document explains, end to end, what was built in the **Fake News Detector** project: how the data was prepared, how the Bidirectional LSTM was designed and trained in the notebook, how it was evaluated, and how it is served through a Streamlit web app. It also contains a data-quality and **leakage audit** that explains why the headline accuracy should be interpreted with care.

Every number in this report is taken from the notebook outputs (training log, confusion matrix, model summary) or recomputed directly from the dataset file shipped in the repository. Analyses that go beyond the original notebook are clearly labeled as *supplementary*.

> **Reading guide**
> Sections 1 to 8 describe the pipeline and its results. Section 9 explains why those results are optimistic. Sections 10 to 12 cover deployment, limitations, and the improvement roadmap.

Inside this report

1 Executive summary
2 Project overview
3 Setup, following the notebook
4 Dataset and EDA
5 Text preprocessing
6 Model architecture
7 Training
8 Evaluation
9 Data-leakage audit
10 Streamlit deployment
11 Limitations and roadmap
12 Reproducibility and appendix

Contents

1. Executive summary . . . 3

2. Project overview and repository layout . . . 4

3. Environment and setup (following the notebook) . . . 5

3.1 Requirements . . . 5

3.2 Option A: run on Google Colab (as authored) . . . 5

3.3 Option B: run locally . . . 5

3.4 Cell-by-cell walkthrough . . . 6

3.5 Run the web app . . . 6

4. Dataset and exploratory analysis . . . 7

5. Text preprocessing . . . 8

6. Model architecture . . . 9

7. Training . . . 10

8. Evaluation . . . 11

9. Data-leakage audit . . . 12

10. Deployment: the Streamlit application . . . 14

11. Limitations and recommendations . . . 15

12. Reproducibility checklist and appendix . . . 16

1. Executive summary

The project builds a binary text classifier that labels a news article as **Real** (label 1) or **Fake** (label 0). Text is converted into fixed-length integer sequences and processed by an **Embedding → Bidirectional LSTM → Dense** network implemented in TensorFlow / Keras. The trained model and tokenizer are exported as pickle files and served by a Streamlit app for live predictions.

Item	Value
Task	Binary text classification (Real = 1, Fake = 0)
Dataset	5,829 articles: 2,834 Fake (48.6%) and 2,995 Real (51.4%); no nulls, duplicates, or empty texts
Input representation	Keras Tokenizer (20,000 words, <OOV>), sequences padded / truncated to 300 tokens
Model	Embedding(128) → BiLSTM(64) → Dropout(0.5) → Dense(32, ReLU) → Dropout(0.3) → Dense(1, Sigmoid); 2,662,977 parameters
Training	Adam, binary cross-entropy, 5 epochs max, EarlyStopping (patience 3); best weights from epoch 2 restored
Evaluation	80 / 20 random split: 4,663 train and 1,166 test articles
Serving	Streamlit web app (`app.py`) loading `Model/model.pkl` and `Model/Tokenzier.pkl`

Key findings

- **Strong headline performance.** Test accuracy 99.66%, precision 99.50%, recall 99.83%, F1 99.67%. Only 4 of 1,166 test articles are misclassified (3 Fake predicted as Real, 1 Real predicted as Fake).
- **Fast convergence and clear overfitting.** Training accuracy reaches 100% by epoch 3 and training loss keeps shrinking, while validation loss is lowest at epoch 2 (0.0204) and then rises. Early stopping correctly restores the epoch-2 weights.
- **The dataset contains strong shortcuts.** 99.6% of Real articles contain the word "Reuters" versus 1.8% of Fake articles, and 84.9% of Real articles open with a "CITY (Reuters) -" dateline (0.0% of Fake). Apostrophe-stripped contractions such as "isn t" appear in 72.5% of Fake and 9.0% of Real articles.
- **Removing "Reuters" alone does not remove the shortcuts.** In a supplementary experiment, a simple TF-IDF + Logistic Regression baseline scores 97.3% (5-fold CV) on the original text and still 96.8% after agency names are stripped, which indicates additional publisher fingerprints.
- **Conclusion.** The model is an excellent demonstration of a deep-learning text pipeline, but it behaves as a *source / style classifier* on this dataset, not as a fact-checker. It should not be used as an authoritative detector.

> **Important**
> The reported 99.66% test accuracy describes performance on this specific dataset, including its leakage. Do not interpret it as real-world fake-news detection accuracy. See Section 9.

2. Project overview and repository layout

The repository is intentionally small and self-contained: the data, the notebook that produces the model, the trained artifacts, the app that serves them, and this report.

```
Fake-news-detector/
├── Dataset/
│   └── news_dataset.csv              # 5,829 labeled articles (columns: x = text, y = label)
├── Model/
│   ├── News_Fake_Detection.ipynb     # Training + evaluation notebook (source of truth)
│   ├── model.pkl                     # Trained Keras BiLSTM (pickled)
│   └── Tokenzier.pkl                 # Fitted Keras Tokenizer (pickled)
├── app.py                            # Streamlit application
├── requirements.txt                  # streamlit, tensorflow, numpy, pandas, scikit-learn
├── Fake_News_Detection_Report.pdf    # This report
├── README.md
└── LICENSE                           # MIT
```

File	Role in the project
`Dataset/news_dataset.csv`	Training and evaluation data. Column x holds the raw article text, column y the label (0 = Fake, 1 = Real).
`Model/News_Fake_Detection.ipynb`	Complete pipeline: load, tokenize, pad, build, train, evaluate, export.
`Model/model.pkl`	The trained network (about 32 MB on disk, 2,662,977 parameters).
`Model/Tokenzier.pkl`	The fitted tokenizer. It must be used with the model so that word indices match the embedding table. (The file name keeps the original spelling.)
`app.py`	Loads both artifacts once, tokenizes and pads user input, and displays the verdict and confidence.
`requirements.txt`	Python dependencies for the app and notebook (see Section 3.1 for one missing package).

End-to-end flow

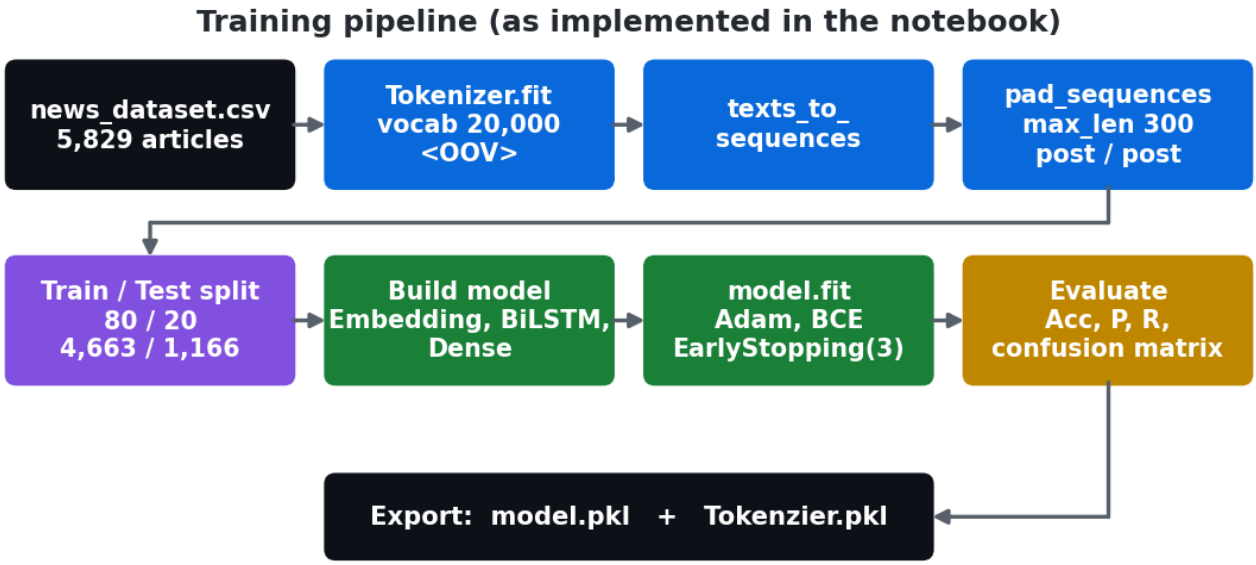


Figure 1. *Training pipeline exactly as implemented in the notebook: data → tokenizer → padded sequences → 80/20 split → BiLSTM → evaluation → pickle export.*

3. Environment and setup (following the notebook)

3.1 Requirements

The repository lists `streamlit>=1.32`, `tensorflow>=2.15`, `numpy>=1.24`, `pandas>=2.0` and `scikit-learn>=1.3`. The notebook additionally imports **matplotlib** for the accuracy and loss plots, which is **not** in `requirements.txt`; install it manually or add it to the file. The notebook was executed on Google Colab with Python 3.12.

3.2 Option A: run on Google Colab (as authored)

- Upload `news_dataset.csv` to the root of your Google Drive (*My Drive*); the notebook reads `/content/drive/MyDrive/news_dataset.csv`.
- Open `Model/News_Fake_Detection.ipynb` in Colab (*File* → *Upload notebook*) and, optionally, switch to a GPU runtime to speed up training.
- Run all cells in order. Approve the Drive permission prompt when `drive.mount` runs.
- After the last cell, download `model.pkl` and `Tokenzier.pkl` from the Colab file browser and place them in `Model/`.

3.3 Option B: run locally

```
git clone https://github.com/<your-username>/Fake-news-detector.git
cd Fake-news-detector

python -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate

pip install -r requirements.txt
pip install matplotlib jupyter       # notebook needs matplotlib

jupyter notebook Model/News_Fake_Detection.ipynb
```

Make two small edits in the notebook before running it locally:

- Delete (or skip) the cell that calls `drive.mount`; it only works in Colab.
- Change the data path to `../Dataset/news_dataset.csv` (the notebook lives in `Model/`). The export cell then writes `model.pkl` and `Tokenzier.pkl` next to the notebook, which is where `app.py` expects them.

3.4 Cell-by-cell walkthrough

#	Notebook step	Key code (abridged)	Result / output
1	Imports	tensorflow, Tokenizer, pad_sequences, LSTM, pandas, numpy, matplotlib	Libraries ready
2	Mount Drive (Colab only)	drive.mount("/content/drive")	"Mounted at /content/drive"
3	Load data	pd.read_csv(".../news_dataset.csv"); df.head()	Columns x (text) and y (label)
4	Select columns	x = df["x"]; y = df["y"]	Features and labels
5	Fit tokenizer	Tokenizer(num_words=20000, oov_token="<OOV>").fit_on_texts(x)	Word index built from all articles
6	Encode text	token.texts_to_sequences(x)	Lists of integer ids
7	Pad / truncate	pad_sequences(x, maxlen=300, padding="post", truncating="post")	Array of shape (5829, 300)
8	Build model	Sequential([Embedding, Bidirectional(LSTM), Dropout, Dense, Dropout, Dense])	Keras warning: *input_length* is deprecated
9	Compile	optimizer="adam", loss=BinaryCrossentropy(), metrics=["accuracy"]	Model ready to train
10	Summary	model.summary()	2,662,977 parameters (10.16 MB)
11	EarlyStopping (v1)	EarlyStopping(patience=15, restore_best_weights=True)	Overwritten by step 14; never used
12	Split	train_test_split(x, y, test_size=0.2)	No random_state, no stratify
13	Check shapes	X_train.shape, X_test.shape, ...	(4663, 300), (1166, 300), (4663,), (1166,)
14	EarlyStopping (v2)	EarlyStopping(patience=3, verbose=1, restore_best_weights=True)	The callback actually used
15	Train	model.fit(X_train, Y_train, epochs=5, validation_data=(X_test, Y_test), callbacks=[early])	Stops after epoch 5, restores epoch 2
16-17	Plot curves	plt.plot(history["accuracy"]) / ["loss"]	Accuracy and loss figures
18	Evaluate	accuracy, precision, recall, confusion_matrix	0.9966 / 0.9950 / 0.9983; [[562, 3], [1, 600]]
19	Export	pickle.dump(model, ...); pickle.dump(token, ...)	model.pkl and Tokenzier.pkl

3.5 Run the web app

```
# run from the repository root so the relative paths resolve
streamlit run app.py
```

The app expects `Model/model.pkl` and `Model/Tokenzier.pkl` relative to the working directory. If either file is missing, it shows an error message instead of crashing. A hosted version is available at the live-demo link in the README.

4. Dataset and exploratory analysis

The dataset is a single CSV with two columns: **x** (the full article text, including headline) and **y** (the label, 0 = Fake, 1 = Real). The classes are well balanced, so no resampling or class weighting was needed.

Property	Value
Rows	5,829
Fake (0) / Real (1)	2,834 (48.6%) / 2,995 (51.4%)
Missing values	0
Duplicate texts	0
Empty texts	0
Unique tokens (Keras tokenization)	57,350
Total tokens	2,505,507

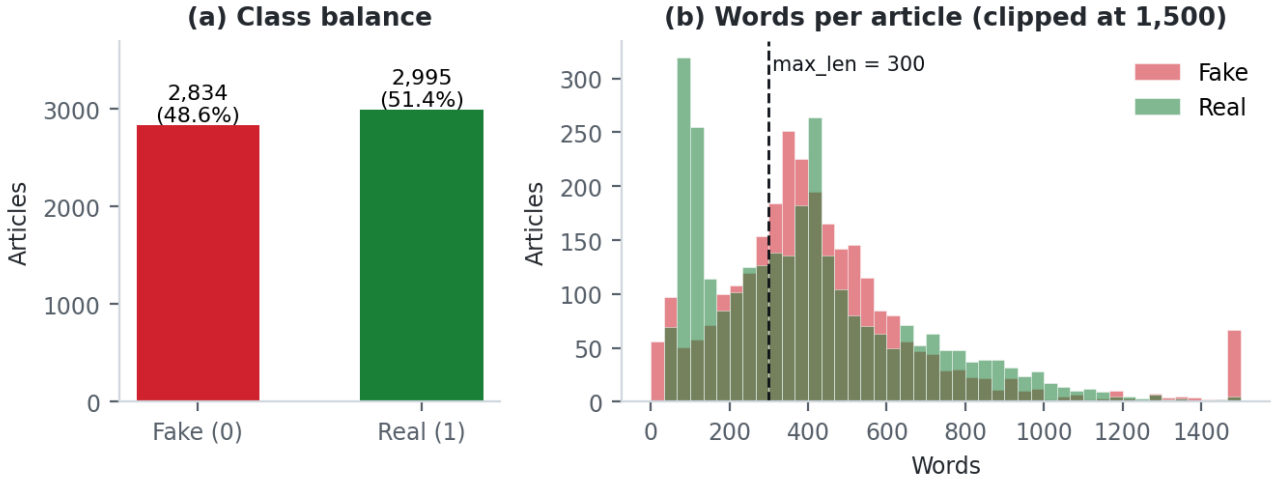


Figure 2. *Left: class distribution. Right: distribution of article length in words for each class; the dashed line marks the 300-token window used by the model.*

Words per article	Fake (0)	Real (1)
Mean	460	401
Median	391	373
Std. deviation	439	274
Minimum	13	39
Maximum	8,389	2,006

Median lengths are similar (391 vs 373 words), but Fake articles have a much heavier tail (up to 8,389 words) and a larger share of very short items, while Real articles show a pronounced cluster of short pieces of roughly 100 words.

Sample records

Label	Article text (first characters)
Real (1)	China backs U.N. call for justice in Yemen, U.S. and Saudis don't. GENEVA (Reuters) - China signaled on W...
Real (1)	Lawmakers aim to delay U.S. ceding control of Internet's management. WASHINGTON (Reuters) - Critics of a ...
Fake (0)	WSJ REPORTER RIPS INTO DEM CANDIDATES For Their Lame Rhetoric On ISIS: “Remarkable displays of unintellig...
Fake (0)	Trump STRANDED, In Full Panic As ANOTHER Performer Drops Out Of Inauguration (DETAILS). President-elect D...

Even in these four rows the stylistic gap is visible: capitalized outrage headlines in the Fake class and wire-style reporting in the Real class. That gap is useful for classification but is not the same thing as truthfulness (see Section 9).

5. Text preprocessing

Preprocessing is deliberately minimal and fully handled by Keras utilities. No stemming, stop-word removal, or manual cleaning is applied, so the network sees the raw writing style, including capitalization effects removed only by lowercasing.

Step	Setting	Effect
Tokenization	Keras Tokenizer, default filters, lowercasing, whitespace split	Punctuation such as !"#$%&()*+,-./:;<=>?@ is replaced by spaces
Vocabulary limit	num_words = 20,000	Only the 20,000 most frequent words keep their own id
Unknown words	oov_token = "<OOV>"	Rarer words map to a single unknown token
Sequence length	maxlen = 300	Every article becomes exactly 300 integers
Padding / truncation	padding = "post", truncating = "post"	Short texts get trailing zeros; long texts keep only the **first** 300 tokens

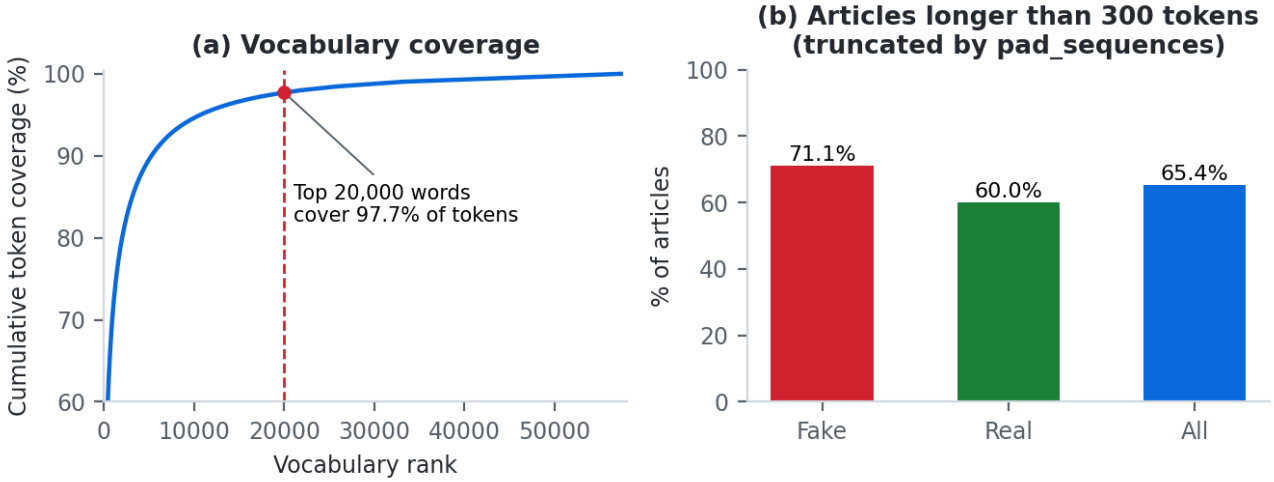


Figure 3. *Left: the 20,000-word vocabulary covers 97.7% of all tokens. Right: share of articles that exceed 300 tokens and are therefore truncated.*

- **Vocabulary coverage is excellent.** The corpus has 57,350 unique tokens, but the top 20,000 account for 97.7% of all occurrences; the 20,000th word still appears 4 times.

- **Most articles are truncated.** 65.4% of articles exceed 300 tokens (Fake 71.1%, Real 60.0%). Because truncation is "post", the model only reads the *beginning* of each article, which is exactly where headlines and wire-service datelines appear.
- **Tokenizer is fitted before the split.** The notebook calls `fit_on_texts` on all 5,829 articles, so the vocabulary also contains words from the later test set. The effect is small (only word ids, no labels), but best practice is to fit on the training portion only.

6. Model architecture

The network is a compact sequence classifier: an embedding layer converts token ids into dense vectors, a bidirectional LSTM summarizes the whole sequence in both reading directions, and a small dense head produces the probability that the article is real.

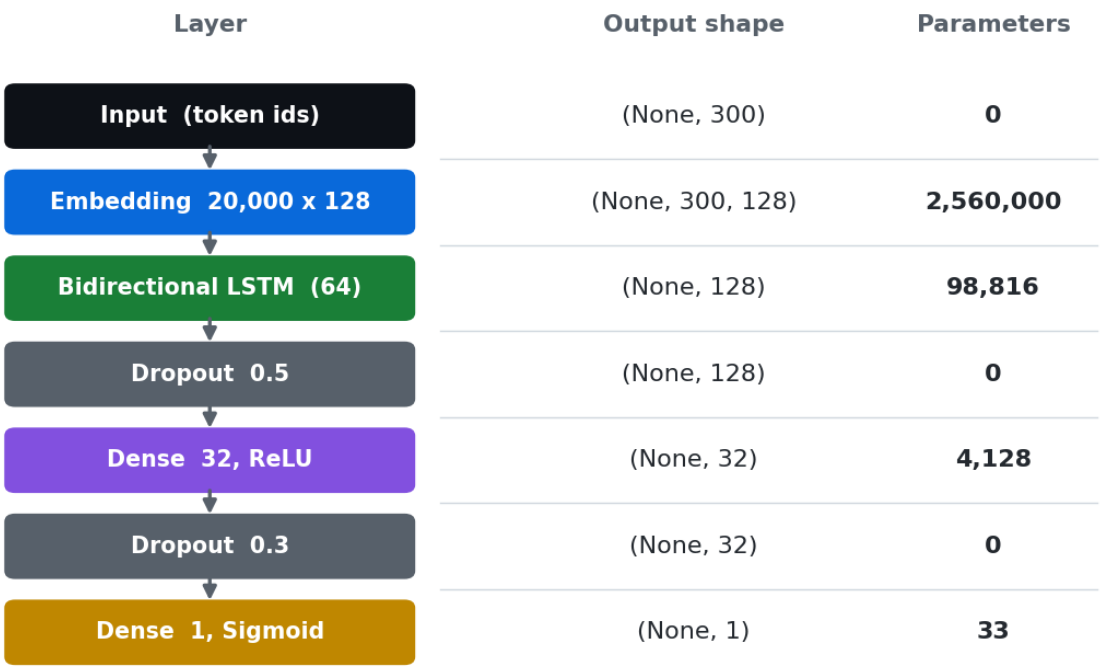


Figure 4. *Layer stack with output shapes and parameter counts (batch dimension shown as None).*

Layer	Parameter formula	Parameters
Embedding	20,000 words × 128 dimensions	2,560,000
Bidirectional LSTM (64)	2 directions × 4 gates × [(128 + 64) × 64 + 64]	98,816
Dense (32, ReLU)	128 × 32 + 32	4,128
Dense (1, Sigmoid)	32 × 1 + 1	33
Total	All layers are trainable	**2,662,977**

Design choices

- **Embedding (128-d).** Learns a dense representation for each of the 20,000 words from scratch; it holds 96% of all parameters.
- **Bidirectional LSTM (64 units per direction).** Reads the sequence left-to-right and right-to-left and concatenates both final states into a 128-d vector, capturing context on both sides of every word.
- **Dropout (0.5 and 0.3).** Regularization against overfitting; stronger after the wide recurrent output and lighter before the output layer.

- **Dense (32, ReLU) and Dense (1, Sigmoid).** A small non-linear head followed by a single sigmoid unit that outputs P(Real) between 0 and 1, matched with binary cross-entropy loss.
- **Padding zeros.** The notebook does not set `mask_zero=True`, so the LSTM also processes padding tokens (about 35% of sequences are shorter than 300 tokens). Masking is a simple improvement for later experiments.

7. Training

Setting	Value
Optimizer	Adam (Keras defaults, learning rate 0.001)
Loss	Binary cross-entropy
Metric	Accuracy
Batch size	32 (Keras default; 146 steps per epoch)
Max epochs	5
Early stopping	Monitor val_loss, patience 3, restore_best_weights = True
Split	80 / 20 random (4,663 train, 1,166 test), no seed, no stratification
Validation data	The same 1,166-article test split
Epoch time	About 62 to 81 seconds on Colab

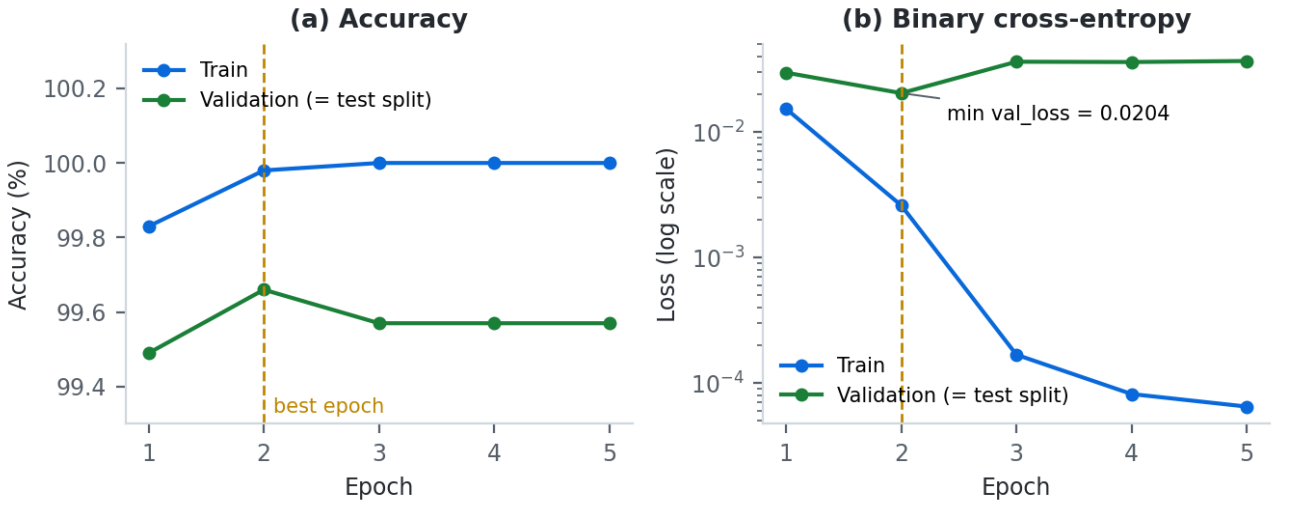


Figure 5. *Training and validation curves from the notebook log. Validation loss is lowest at epoch 2 (dashed line) and increases afterwards while training loss keeps falling.*

Epoch	Train acc.	Train loss	Val. acc.	Val. loss	Time (s)
1	0.9983	0.0153	0.9949	0.0297	74
2 (best)	0.9998	0.0026	**0.9966**	**0.0204**	62
3	1.0000	0.000168	0.9957	0.0363	64
4	1.0000	0.000082	0.9957	0.0361	81
5	1.0000	0.000065	0.9957	0.0368	81

- **Overfitting is visible after epoch 2.** Training loss falls by more than two orders of magnitude, while validation loss increases from 0.0204 to about 0.037.
- **Early stopping did its job.** No improvement for epochs 3, 4 and 5 triggered the stop, and the weights of epoch 2 were restored; these are the weights that were exported.

- **The validation curve is not independent.** Because validation data equals the test split, choosing the best epoch uses information from the final test set (see Section 11).

8. Evaluation

Evaluation uses the restored epoch-2 model on the 1,166 held-out articles. The positive class is **Real (1)**, so precision answers "of the articles called Real, how many were Real?" and recall answers "of the Real articles, how many were found?".

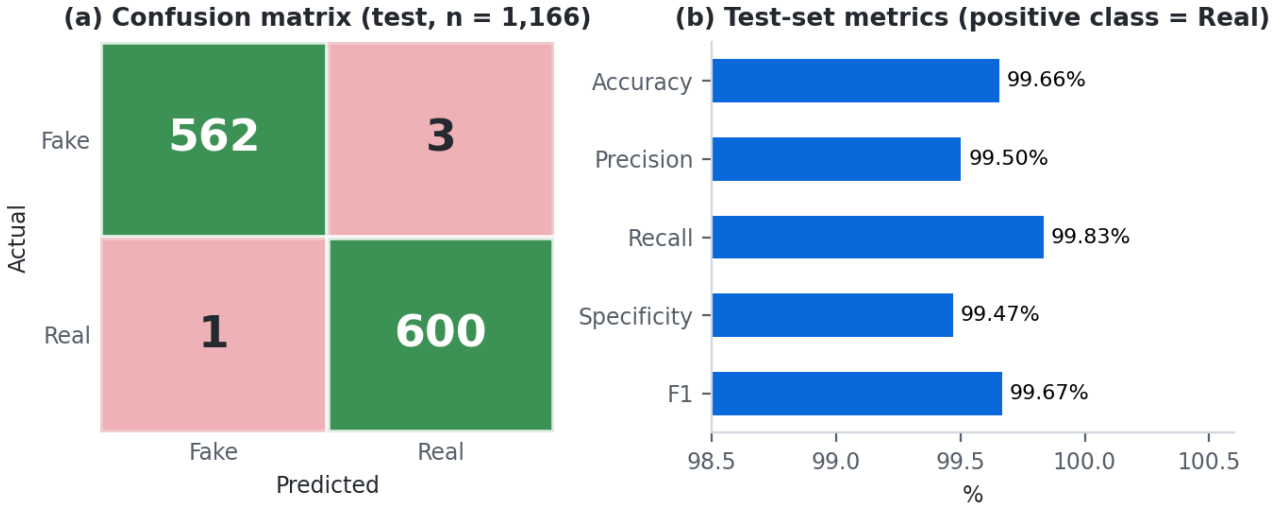


Figure 6. *Left: confusion matrix on the test set. Right: metrics derived from it. All values are above 99%.*

Metric	Formula	Computation	Value
Accuracy	(TP + TN) / N	(600 + 562) / 1,166	99.66%
Precision	TP / (TP + FP)	600 / 603	99.50%
Recall (sensitivity)	TP / (TP + FN)	600 / 601	99.83%
Specificity	TN / (TN + FP)	562 / 565	99.47%
F1 score	2PR / (P + R)	harmonic mean of P and R	99.67%
Train accuracy	recomputed in inference mode	4,663 / 4,663	100.00%

- **Errors are rare and balanced.** Three Fake articles were classified as Real and one Real article as Fake, which gives 4 errors out of 1,166 (0.34%).
- **Train accuracy of 100% versus 99.66% on test** is a small generalization gap, but it is a gap between two samples of the same distribution (same publishers, same period, same formatting).
- **The four misclassified articles were not inspected in the notebook.** Reading them is a cheap, informative next step: they likely reveal which shortcuts the model relies on.

9. Data-leakage audit

Leakage occurs when label information is encoded in features that will not be available (or will not mean the same thing) in real use. Here, the label is strongly encoded in *where the text came from* and *how it was formatted*.

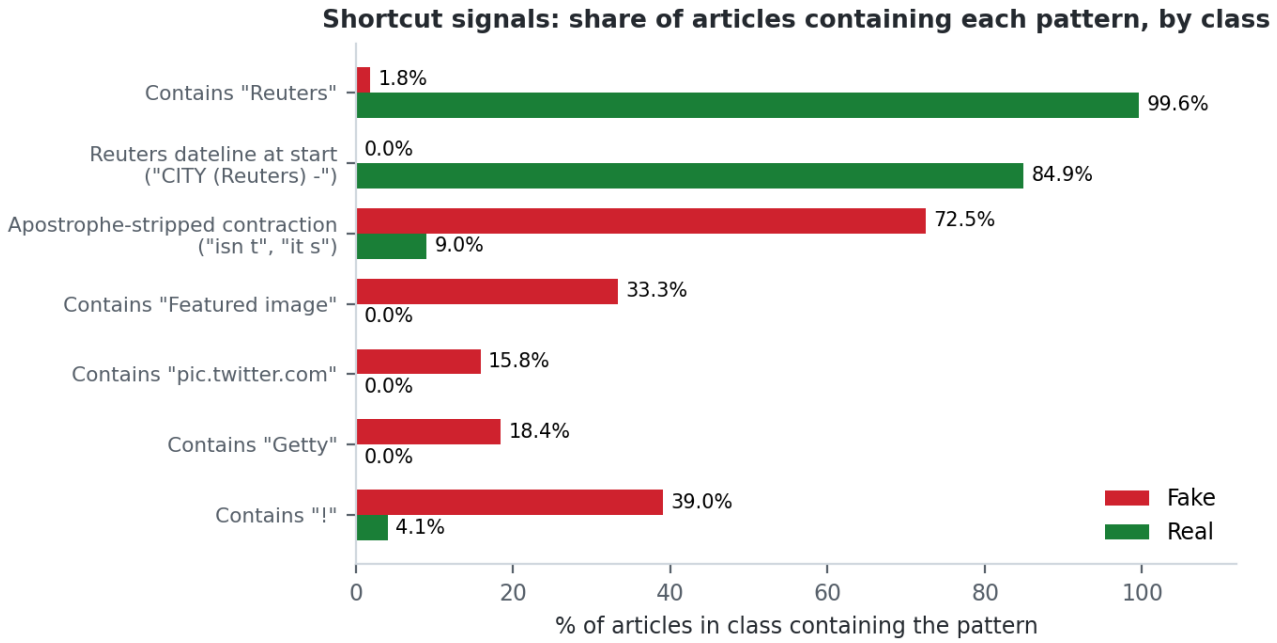


Figure 7. *Share of articles in each class that contain a given pattern. Several patterns appear almost exclusively in one class.*

Signal	Fake (0)	Real (1)	Reading
Contains "Reuters"	1.8%	99.6%	Source name acts as a label proxy
Starts with "CITY (Reuters) -"	0.0%	84.9%	Dateline sits inside the first 300 tokens
Apostrophe-stripped contractions	72.5%	9.0%	Text-processing artifact ("isn t", "it s")
Contains "Featured image"	33.3%	0.0%	Website boilerplate
Contains "pic.twitter.com"	15.8%	0.0%	Embedded-tweet residue
Contains "Getty"	18.4%	0.0%	Image-credit residue

What the vocabulary says

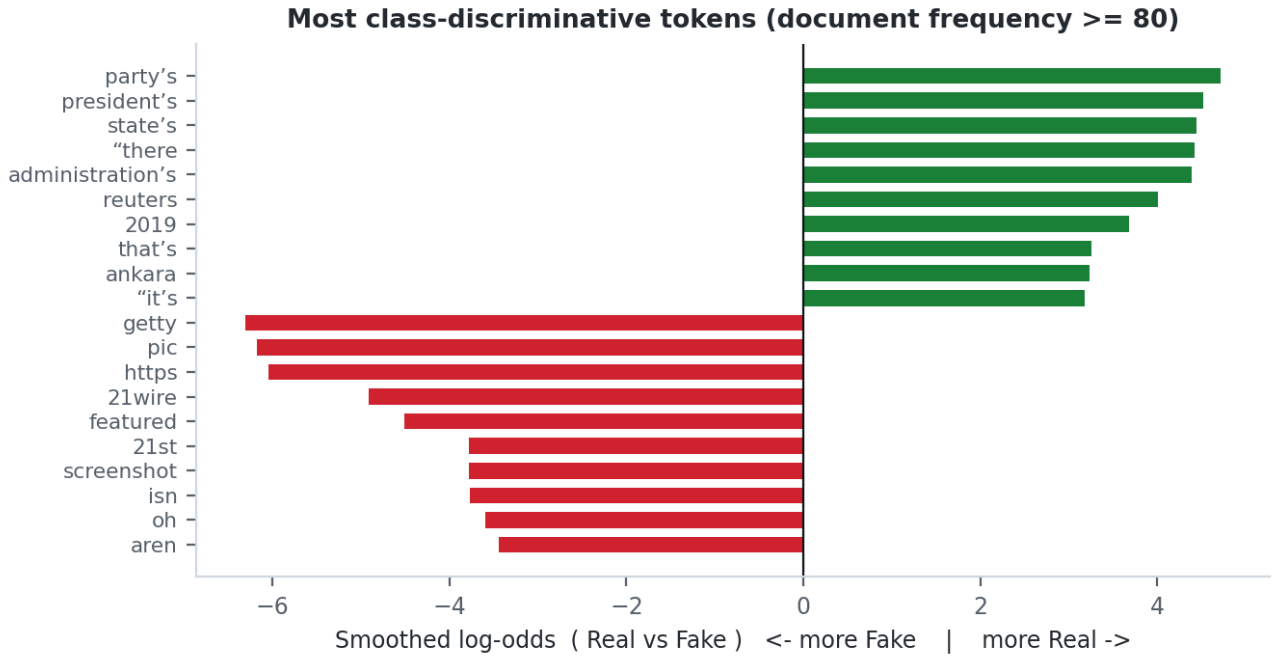


Figure 8. *Tokens with the largest difference in document frequency between classes (smoothed log-odds). Many are formatting artifacts, not topics.*

- **Fake-side tokens** such as "getty", "pic", "https", "featured" and "screenshot" come from images, links, and embedded media; "isn", "aren", "couldn" are the pieces left when apostrophes were stripped from contractions.
- **Real-side tokens** include "reuters" and words attached to typographic apostrophes or quotation marks (for example "president's", ""the"), which reflect a different text-encoding pipeline rather than content.

Supplementary experiment: does removing "Reuters" fix it?

To test whether the agency name alone explains the score, a lightweight baseline (TF-IDF with 20,000 features and Logistic Regression, stratified 5-fold cross-validation, seed 42) was trained on two versions of the text: the original, and a version with Reuters datelines and the word "Reuters" removed. *This baseline is not the BiLSTM and is not part of the notebook; it is used only to probe the data.*

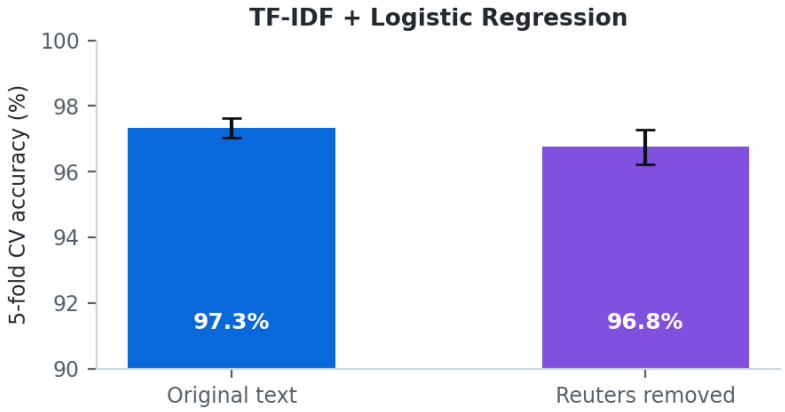


Figure 9. *Accuracy of a TF-IDF + Logistic Regression baseline with and without agency tokens. Error bars show one standard deviation across folds.*

Interpretation

Accuracy drops by only about half a point (97.3% → 96.8%). The dataset therefore separates by many publisher fingerprints at once, so deleting one token is not enough. A trustworthy evaluation needs **source-disjoint** splits (train and test from different publishers) and normalized text.

10. Deployment: the Streamlit application

`app.py` turns the exported artifacts into an interactive tool. Inference mirrors training exactly: same tokenizer, same `maxlen=300`, same `post / post` padding.

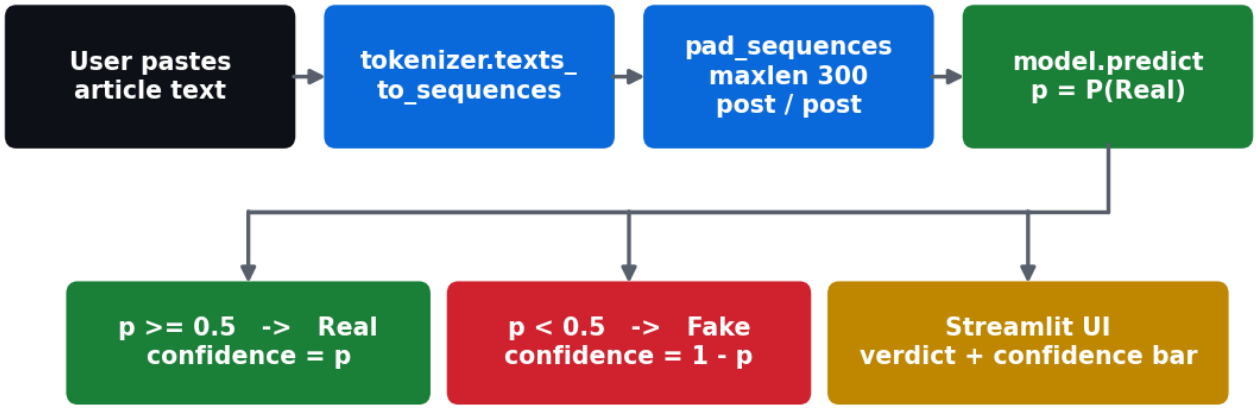


Figure 10. *Inference path inside the app, from pasted text to a verdict with a confidence score.*

Component	Implementation	Purpose
Configuration	MODEL_PATH = "Model/model.pkl", TOKENIZER_PATH = "Model/Tokenzier.pkl", MAX_LEN = 300	Constants that must match training
Artifact loading	@st.cache_resource load_artifacts() using pickle	Loads model and tokenizer once and reuses them
Prediction	predict(text, model, tokenizer)	texts_to_sequences → pad_sequences → model.predict
Decision rule	p ≥ 0.5 → Real, otherwise Fake	Fixed threshold; confidence = p or 1 − p
Input	selectbox with two examples + text_area (220 px)	Wire-service style and clickbait style samples
Output	st.success / st.error, st.metric, st.progress	Verdict, confidence percentage, progress bar
Details	Expander with the raw probability and the rule	Transparency about the score
About	Expander with architecture, data size, accuracy and disclaimer	Model card inside the app
Error handling	FileNotFoundError → st.error; empty text → st.warning	The app never fails silently

- **Confidence is not calibrated truth.** It is the sigmoid output of an over-confident model (training loss near zero), so 99% means "very similar to the training pattern", not "99% likely to be true".
- **Version compatibility.** The artifacts are pickled Keras objects; loading requires a compatible TensorFlow / Keras version. Pin the training versions or switch to the native `.keras` format.
- **Security.** Never unpickle files from untrusted sources, because pickle can execute arbitrary code during loading.

11. Limitations and recommendations

#	Issue	Impact	Recommended fix	Priority
1	Reuters wire-service leakage (99.6% vs 1.8%)	Inflated accuracy; model can read the source instead of the content	Strip agency names and datelines; retrain; report with and without	High
2	Other publisher fingerprints (apostrophe artifacts, image credits, links)	Shortcuts survive Reuters removal (baseline still 96.8%)	Normalize punctuation and boilerplate; source-disjoint train / test split	High
3	Narrow, undocumented data (English, limited time window, unknown provenance)	Unknown generalization to other outlets and newer events	Add a data card; evaluate on out-of-distribution articles	High
4	Test split used as validation data	Best-epoch selection leaks test information	Use train / validation / test	Medium
5	Head-only truncation (65.4% of articles cut at 300 tokens)	Tail ignored; datelines always in view	Try head + tail windows, longer inputs, or chunking	Medium
6	Pickle artifacts	Security and version-compatibility risk	Export .keras model and tokenizer JSON; pin versions	Medium
7	Unseeded, non-stratified split	Results vary between runs	Set random_state and stratify = y	Low
8	Tokenizer fitted on all data	Minor vocabulary leakage into test	Fit on training portion only	Low
9	matplotlib missing from requirements; versions unpinned	Fresh installs can fail or behave differently	Add and pin dependencies	Low
10	Style classifier, not a fact-checker	Cannot verify claims	State clearly; combine with retrieval / fact-checking tools	Inherent

Roadmap

Phase	Goal	Concrete tasks
1. Valid evaluation	Measure what the model really learns	Cleaning function for agency names, dateline and punctuation artifacts; ablation with and without them; three-way stratified split with a fixed seed; source-disjoint test set
2. Better models	Stronger, honest baselines	TF-IDF + Logistic Regression baseline; mask_zero and attention pooling; fine-tuned transformer (DistilBERT / RoBERTa); cross-validation with confidence intervals; ROC / PR and calibration curves
3. Engineering	Reproducible and safe	src/ package with CLI; .keras + tokenizer JSON; tests and GitHub Actions CI; Dockerfile; pinned requirements
4. App	More transparent UX	Word-level explanations (LIME / SHAP); warnings for short or non-English text; batch prediction from CSV

12. Reproducibility checklist and appendix

Item	Status
Dataset included in repository	Yes (Dataset/news_dataset.csv)
Training code	Yes (notebook; Colab paths need a local edit)
Random seed fixed	No (split is unseeded; weights initialized randomly)
Exact library versions recorded	No (requirements use lower bounds; notebook ran on Python 3.12 / Colab)
Trained artifacts included	Yes (model.pkl, Tokenzier.pkl)
Metrics reproducible to the decimal	Not guaranteed, because of the unseeded split and weight initialization

Glossary

Term	Meaning
BiLSTM	Bidirectional Long Short-Term Memory: two LSTMs read the sequence in opposite directions and their outputs are concatenated.
Embedding	Trainable lookup table mapping each word id to a dense vector.
OOV	Out-of-vocabulary: a word outside the 20,000 kept by the tokenizer.
Early stopping	Stop training when validation loss stops improving and restore the best weights.
Data leakage	Information in the features that reveals the label but will not generalize.
Source-disjoint split	Train and test sets drawn from different publishers.

References

- Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Schuster, M. and Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681.
- Srivastava, N. et al. (2014). Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
- Kingma, D. P. and Ba, J. (2015). Adam: a method for stochastic optimization. *International Conference on Learning Representations*.
- Chollet, F. et al. (2015). Keras. https://keras.io

Disclaimer

This project and report are provided for educational and demonstration purposes. The model analyzes writing patterns and formatting; it does not verify facts. Always check important news with trusted, independent sources.